

ShoreGuard - Technical Documentation

Technical Documentation

1. Overview

ShoreGuard is a beach volunteer coordination and emergency alert system designed for community safety volunteers and beach facilities. The application separates users into two main account types:

1. **Volunteer (Lifeguard/Community Volunteer)**
2. **Beach Facility (Beach Management)**

2. Implementation Status (27 May 2026)

- Fully Implemented: Login, Create Account, Volunteer sign-up flow, Beach Facility sign-up flow, basic navigation.
- Partially Implemented: SharePoint data connections, roster viewing, alert creation.
- Not Implemented: Full login validation against SharePoint, dynamic filtering, alert response logic.

The system manages:

1. Volunteer registrations
2. Beach facility registrations
3. Daily volunteer beach bookings
4. Beach rosters
5. Emergency volunteer alerts

3. User Roles

Role	Description
Lifeguard (Volunteer)	Active user that claims patrol shifts, uploads certified documents, and receives urgent alerts from nearby beaches.

Beach Facility (Coordinator)	Posts shifts, sends out urgent alerts, assigns volunteers to the rosters, and verifies users' certifications.
------------------------------	---

4. The frontend contains:

- Login system
- Volunteer sign-up flow
- Beach facility sign-up flow
- Volunteer: beach and slot booking for lifeguarding
- Volunteer: emergency alert response
- Beach facility: roster pages
- Beach facility: emergency alert pages

5. Backend Components

The Shore Guard backend is designed using Microsoft SharePoint Lists as the primary data storage system. SharePoint was selected because it provides:

- Cloud-based centralized storage
- Secure user data management
- Easy CRUD operations (Create, Read, Update, Delete)
- Real-time data synchronization
- Filtering and searching functionality
- Integration with Microsoft ecosystem tools
- Scalability for future expansion
- Multi-user accessibility for beach facilities and administrators

SharePoint List Structure

– The application uses three primary SharePoint Lists.

1. Lifeguard Details List

This SharePoint List stores all volunteer-related information.

Purpose

- Store volunteer account details
- Store volunteer certification
- Store booking details
- Provide roster data to beach facilities

Technical Operations

Operation	Backend Action
Volunteer Sign Up	Creates new SharePoint list item
Volunteer Login	Queries SharePoint list using email/password
Activity Selection	Updates volunteer record
Booking Confirmation	Updates booking status
Roster Generation	Retrieves volunteers matching beach and time slot

SharePoint Fields

Field Name	Data Type
Username	Single Line Text
Email	Single Line Text
Password	Single Line Text
Contact Number	Single Line Text
Certification	Image/Attachment
Selected Beaches	Multi-line text
Slot Today	Single Line Text
Beach Today	Single Line Text

2. Beach Details List

This SharePoint List stores beach facility information.

Purpose

- Store beach facility accounts
- Store beach operational timing

Technical Operations

Operation	Backend Action
Beach Sign Up	Creates SharePoint list item
Beach Login	Queries SharePoint list
Roster View	Retrieves matching volunteers

SharePoint Fields

Field Name	Data Type
Beach Name (Title)	Single Line Text
Username	Single Line Text
Email	Single Line Text
Password	Single Line Text
Contact Number	Single Line Text
Opens	Single Line Text
Closes	Single Line Text

3. Beaches List

This SharePoint List stores beach names and volunteer requirements.

Purpose

- Store available beach locations
- Track number of volunteers needed per beach during an emergency

Technical Operations

Operation	Backend Action
-----------	----------------

Beach Dropdown	Queries SharePoint list for beach names
Raise Alert	Updates volunteer requirement for the beach

SharePoint Fields

Field Name	Data Type
Beach Names	Single Line Text
Number Of Volunteers	Number
Alert Reason	Single Line Text

6. Authentication and Validation Logic

Login Validation Process

– The login system uses conditional validation logic.

Volunteer Authentication Logic

1. User enters email and password.
2. System queries the Lifeguard Details SharePoint List.
3. Backend checks whether:

Entered Email == Stored Email

AND

Entered Password == Stored Password

***For the same user**

4. If validation succeeds:
 - User session is authenticated.
 - User is redirected to Volunteer Flow.
5. If validation fails:
 - Error message is returned.

Beach Facility Authentication Logic

1. User enters email and password.
2. System queries the Beach Details SharePoint List.
3. Backend checks whether:

Entered Email == Stored Email

AND

Entered Password == Stored Password

4. If validation succeeds:
 - User is redirected to Beach Facility Flow.
5. If validation fails:
 - Error message is displayed.

7. Role-Based Navigation Logic

The application uses role-based routing.

Volunteer Routing

If account type = Volunteer:

Login → Volunteer Pages

Volunteer users can:

- Select beach for lifeguarding
- Book volunteer slots
- Receive emergency alerts

Beach Facility Routing

If account type = Beach Facility:

Login → Beach Dashboard

Beach facility users can:

- View volunteer rosters
- Raise emergency alerts

8. Dynamic Roster System

The roster system dynamically displays volunteers depending on:

- Selected beach
- Selected time slot

9. Roster Generation Logic

The backend automatically:

1. Searches volunteer records.
2. Filters volunteers by beach.
3. Filters volunteers by time slot.
4. Displays matched volunteers in the roster page.

Example:

IF Volunteer.SelectedBeach == Cronulla Beach

AND Volunteer.TimeSlot == 8:00 to 10:00

THEN Display on Cronulla Morning Roster

10. Emergency Alert Logic

– The emergency alert system allows beach facilities to request additional volunteers during emergencies.

Alert Storage Process

When “Raise Alert” is clicked:

1. Logged-in beach name is automatically identified in the beaches list.
2. System matches:
 - Beach name
3. System stores:
 - Number of volunteers required
 - Reason For Alert
4. Volunteers receive notification.

11. Security Considerations

The system includes:

- Role-based authentication

- Separate data lists for volunteers and beach facilities
- Controlled login validation
- Centralized cloud storage
- Data separation between account types

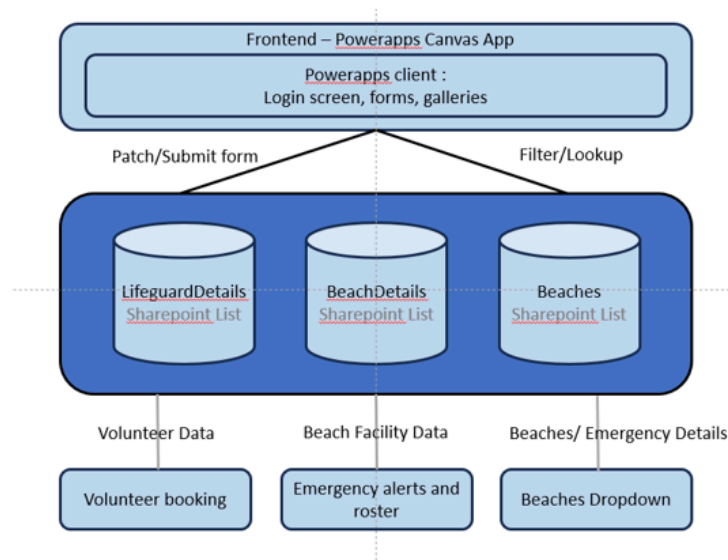
Future improvements may include:

- Password encryption
- Multi-factor authentication
- Role-based admin permissions
- Real-time push notifications

12. Scalability Considerations

The architecture supports future expansion such as:

- GPS on beaches to pin-point exact location of emergency on the beach
- Larger volunteer databases
- Additional emergency categories
- Flexible shift scheduling
- Another community login for anyone to update an emergency on the beach



The diagram above illustrates the high-level architecture of the ShoreGuard data backend.

Core Components:

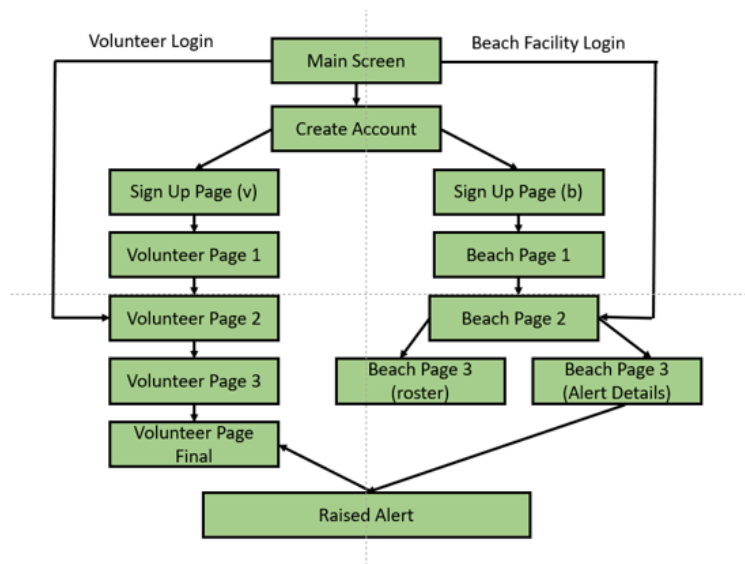
- **Frontend:** Power Apps Canvas App (all user screens and logic)
- **Backend:** Three Microsoft SharePoint Online Lists acting as the primary databases:
 - **LifeguardDetails List** – Stores all volunteer-related data (registrations, certifications, beach & time slot selections, booking status).
 - **BeachDetails List** – Stores beach facility accounts, operating timings, emergency alert requests, and roster information.
 - **Beaches List** – Stores a list of all beaches and the number of volunteers needed during an emergency.

Data Operations:

- **Create:** Patch() or SubmitForm() functions are used to write data from sign-up forms and alert creation screens into the respective SharePoint Lists.
- **Read:** Filter() and LookUp() functions are used to retrieve data for roster viewing (VolunteersList) and alert details.

Current Status: SharePoint connectors have been added to the app. Form submissions during the Volunteer and Beach Facility onboarding flows now persist data into the SharePoint Lists. Full dynamic querying and bidirectional updates are still in progress.

UI FLOW DIAGRAMS



Requirements Traceability

ID	Requirement Description	Implemented In	Status	Comments
FR1	Volunteers can create profile and register availability	Sign Up Page(v), Certification Upload, Choose Beach & Time	Implemented	Full onboarding flow completed
FR2	Beach Facility can create account and set timings	Sign Up Page(b), beach-timings	Implemented	Beach onboarding flow completed
FR3	Volunteers can select beach and time slot for booking	Choose Beach & Time, booking confirmation	Implemented	Core booking steps working
FR4	Volunteer receives confirmation after booking	booking confirmation, volunteer - congrats	Implemented	Final congrats screen reached
FR5	Beach Facility can view roster	CheckRoster	Implemented	Roster viewing functional
FR6	Beach Facility can raise emergency alert	RaiseAlert, Raised Alert	Implemented	Alert creation screen active
FR7	Users can view alert details and	Raised Alert, Alert Details	Partially Implemented	UI present; backend

	respond (Accept/Decline)			response logic pending
AC1	Role-based routing from Login Screen	Login Screen (Volunteer / Beach Facility buttons)	Implemented	Correct navigation based on role
AC2	Volunteer data stored during sign-up	Sign Up Page(v) → Lifeguard Details List (planned)	Partially Implemented	UI flow done; full SharePoint Patch pending
AC3	Certification upload functionality	Certification Upload	Implemented	Upload screen ready
AC4	Dynamic roster based on beach and time slot	VolunteersList (filtered view)	Partially Implemented	UI ready; full filtering logic pending

Non-Functional Requirements

NFR1 — Usability: Core tasks (shift claim, alert accept) are achievable in ≤ 3 taps from the home screen. Verified via manual walkthrough during testing.

NFR2 — Security: Volunteer data is stored in SharePoint with M365 role-based access control. Certification documents are stored in a restricted SharePoint document library accessible only to the volunteer and coordinators.

NFR3 — Availability: The app depends on Microsoft 365 infrastructure. SharePoint Online and Power Apps have a published SLA of 99.9% uptime, exceeding the required 99.5%.

Known Limitations

Limitation	Impact	Workaround
------------	--------	------------

Power Apps Test Studio cannot test modern controls (galleries with nested buttons)	Automated tests cannot cover shift claim and alert accept flows end-to-end	Classic controls used for tested screens; modern controls used for non-tested display-only galleries
Delegation limit: 500 records default for <code>Filter()</code>	May miss shifts or volunteers if data exceeds 500 rows	SharePoint connector delegation increased to 2000 in settings; total dataset expected < 500 for patrol season
No real-time push notifications in Power Apps canvas	Urgent alerts are not instantly pushed to device	Power Automate sends email + Teams notification on alert creation; in-app gallery auto-refreshes every 30s via Timer control
File upload (Attachments control) max 10 MB	Large certification PDFs may be rejected	Users instructed to compress PDFs below 10 MB; documented in user guid